Datamecum

Modelos fundacionales en visión artificial

Tema 4: DINO y representaciones visuales

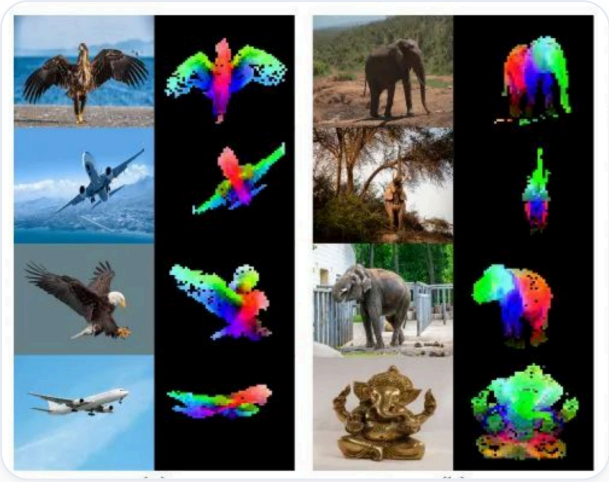
Día 2 del curso

Web del curso	datamecum.sergiovillanuevalopez.com
Práctica	Cuaderno 4 en Colab (4_DINO.ipynb)
Profesor	Sergio Villanueva · Datamecum

4. DINO (v1, v2, v3): ver sin etiquetas

Representaciones visuales generales para muchas tareas.

DINO (self-DIstillation with NO labels) y sus sucesores aprenden representaciones visuales directamente de los píxeles, sin etiquetas humanas, y esas representaciones sirven después para tareas muy distintas.



¿Por qué autosupervisado?

El aprendizaje autosupervisado permite entrenar modelos de visión con muchas más imágenes de las que nadie podría etiquetar.

Aprendizaje supervisado

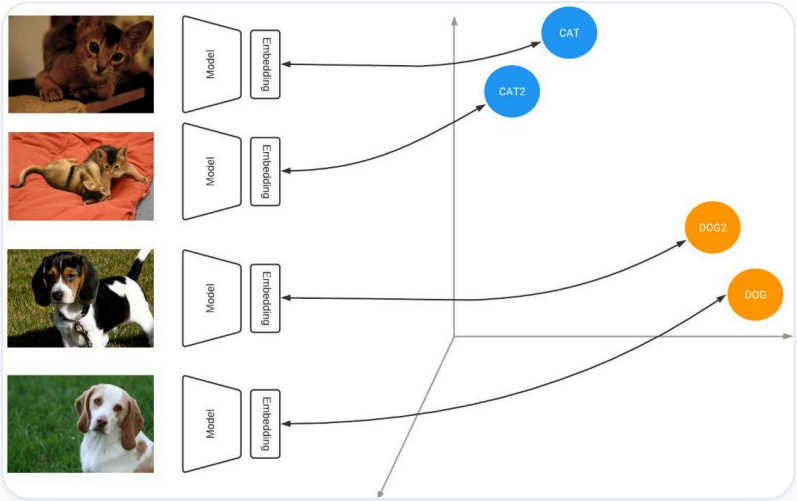
- Necesita etiquetas manuales de expertos.
- Costoso y lento de producir.
- Limitado a categorías predefinidas.
- No escala bien a grandes volúmenes de datos.

Ejemplo: ImageNet

- 14 millones de imágenes en más de 20.000 categorías.
- La versión de competición usa 1.000 categorías y 1,3 millones de imágenes.
- Unos tres años de etiquetado.
- Decenas de miles de personas en Mechanical Turk.

Aprendizaje autosupervisado

- No requiere etiquetas manuales.
- Aprende de la estructura interna de los datos.
- Escala a miles de millones de imágenes: DINOv3 usó 1.689 millones.
- Genera representaciones más generales y transferibles.



Cómo se aprende sin etiquetas

El modelo genera sus propias señales de supervisión desde los datos.

Contrastive learning

- Imágenes similares → embeddings cercanos.
- Imágenes diferentes → embeddings lejanos.

Masked prediction

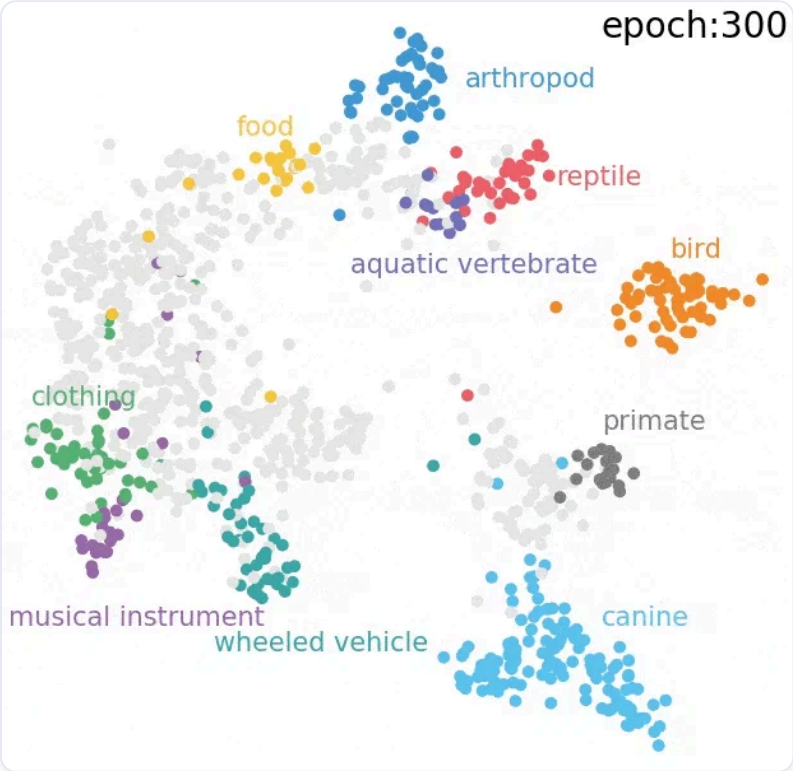
- Ocultar partes de imagen.
- Predecir partes ocultas.

Rotation prediction

- Rotar imagen.
- Predecir ángulo de rotación.

Clustering

- Agrupar features similares sin saber qué representan.



Autodestilación

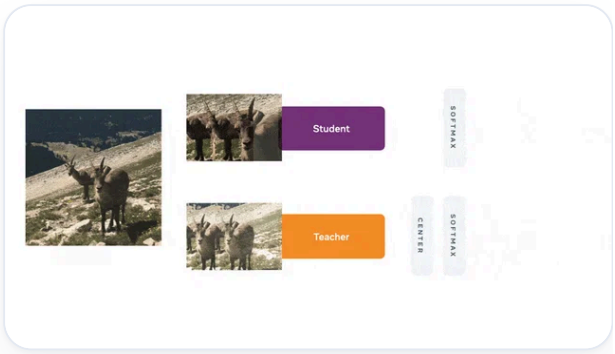
DINO entrena una red enseñándole a predecir la salida de su versión más estable, la red maestra (teacher), sobre diferentes vistas de la misma imagen. Así, distintas vistas de una misma imagen acaban con representaciones coherentes.

Red estudiante (student network)

- Es la que aprende.
- Recibe todas las vistas: las globales y los recortes locales pequeños, que solo ve ella.

Red maestra (teacher network)

- Es la misma red que la estudiante, pero cambia más despacio.
- No aprende por su cuenta: va copiando poco a poco a la estudiante.
- Recibe solo las vistas globales de la imagen.



La estudiante tiene que reproducir lo que la maestra predice para la imagen entera aunque ella solo vea un trozo. Para conseguirlo aprende representaciones que capturan de qué trata la imagen, y en todo el proceso no interviene ninguna etiqueta.

El papel de la red maestra

Las dos redes comparten arquitectura, pero no aprenden igual.

¿Qué función cumple la red maestra en DINO?

- A Ofrecer objetivos para que aprenda la estudiante.
- B Etiquetar manualmente cada imagen.
- C Generar texto sobre la imagen.

Respuesta A. Exacto: la maestra proporciona una referencia durante el entrenamiento.

La maestra no se entrena por backpropagation: sus pesos son un promedio móvil de los de la estudiante. Esa estabilidad es lo que da un objetivo consistente y evita el colapso de las representaciones.

Mapas de atención de DINO

Al entrenar Vision Transformers (ViT) con el algoritmo DINO, el modelo aprende por su cuenta a separar el objeto principal del fondo.

El descubrimiento: segmentación autoaprendida. El modelo aprende a separar objetos sin anotaciones humanas y sin ninguna función de pérdida de segmentación a nivel de píxel.



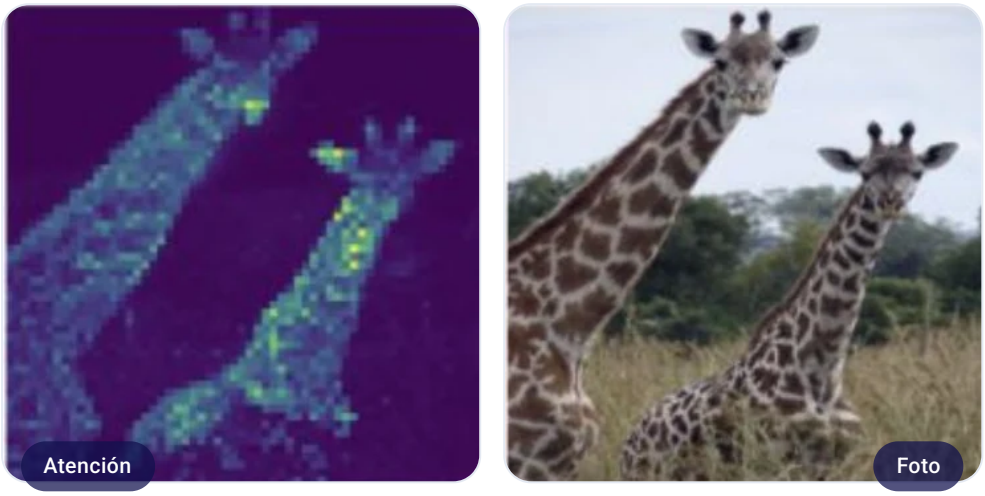
Por qué funciona: la autoatención (self-attention).

- **Visión global:** cada parte de la imagen construye su comprensión procesando y relacionándose con otras ubicaciones, incluso las más distantes.
- **Comprensión de escena:** así cada parche lleva información del resto de la imagen, no solo de su entorno.
- **Segmentación por atención:** al analizar los mapas de atención internos del modelo, se observa que estos se alinean directamente con regiones semánticas coherentes, es decir, el modelo «separa» los objetos automáticamente.

Los mapas de atención son una señal interna del modelo y se alinean a menudo con objetos, pero no son una máscara de segmentación garantizada ni validada píxel a píxel.

Atención sobre la imagen

Desliza para pasar de la imagen al mapa de atención. El modelo se entrenó sin etiquetas.



Dos animales del mismo tipo y el mapa los separa a los dos, con más intensidad en cuellos y cabezas.

Los recortes salen de la figura original del artículo de DINO, que muestra la atención del token de clase en la última capa de un ViT entrenado sin etiquetas. El mapa tiene la resolución de los parches, de ahí el aspecto cuadriculado: con parches de 8 píxeles se ve más fino que con parches de 16. Conviene recordar que es una señal interna del modelo y no una máscara validada píxel a píxel, así que sirve para entender qué mira y no como salida de segmentación.

DINO: identificación de copias de imágenes

DINO detecta copias de una imagen aunque nadie lo entrenó para eso, también cuando están emborronadas, llevan elementos añadidos o se han impreso y escaneado. En el banco de pruebas Copydays supera a Multigrain, un método entrenado específicamente para recuperar imágenes. Sirve para encontrar fotos reutilizadas o duplicadas en un catálogo.



Evolución de la familia DINO

DINO, acrónimo de «self-DIstillation with NO labels», es la familia de referencia del aprendizaje autosupervisado en visión.

1

2021 · DINO

Entrena un ViT sin etiquetas y demuestra que su atención separa objetos sin supervisión explícita.

2

2023 · DINOv2

142 millones de imágenes curadas (LVD-142M) y mejoras de entrenamiento. Sus features funcionan sin ajuste en muchas tareas.

3

2025 · DINOv3

1.689 millones de imágenes y un modelo de 7.000 millones de parámetros. Gram anchoring evita que las features densas se degraden en entrenamientos largos.

> DINOv2 es el backbone de RF-DETR (tema 1) y de Depth Anything (tema 6). Ojo con dos confusiones habituales: SAM no usa DINO, y el DINO de Grounding DINO es un detector que solo comparte el nombre.



Capacidades emergentes de DINO

Descubrimientos importantes

Además de aprender representaciones, DINO hace bien cuatro cosas para las que nunca se entrenó.

Segmentación semántica

Los mapas de atención de DINO actúan como máscaras de objetos, distinguiendo el primer plano del fondo. Identifica objetos coherentes en la escena sin ninguna anotación durante su entrenamiento.

Correspondencias

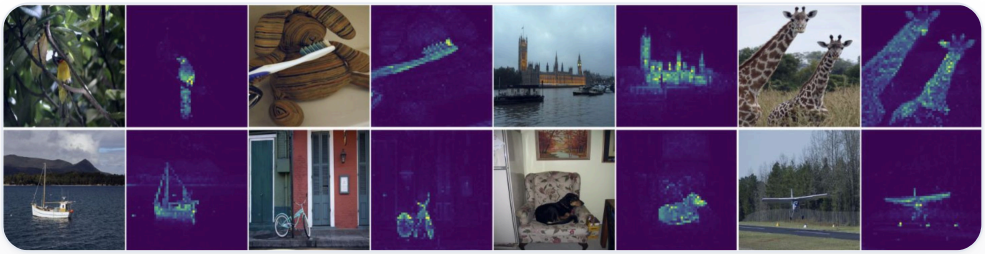
Es capaz de encontrar puntos equivalentes y coherentes entre diferentes vistas de una imagen o incluso entre imágenes distintas. Esta capacidad surge sin haber sido supervisado nunca para tareas de correspondencia.

Detección de objetos

Los heads de atención de DINO se focalizan naturalmente en objetos salientes dentro de la imagen. Esto ocurre sin que el modelo haya visto una sola bounding box durante su fase de preentrenamiento.

Clasificación

Un clasificador k-NN sobre sus features llega al 77,4 % en ImageNet con un ViT-B/8, sin haber visto una sola etiqueta de clase al entrenar.



DINO como extractor de características

El uso más habitual de DINO: convertir cualquier imagen en vectores útiles sin anotar nada.

Para qué

Sacar un vector de cualquier imagen sin anotar nada y usarlo después en otra tarea.

Dos tipos de vector

Uno que resume la imagen entera (el token CLS) y uno por parche, que conserva dónde está cada cosa.

Por qué sirve en tu dominio

DINOv2 vio 142 millones de imágenes sin etiquetas. Sus vectores suelen funcionar también con fotos que nunca vio, como piezas de una fábrica.

Qué se hace con ellos

- Buscar imágenes parecidas.
- Agrupar sin etiquetas.
- Clasificar con pocos ejemplos (k-NN).
- Detectar anomalías.



Similitud entre embeddings

La similitud coseno compara la dirección de dos embeddings normalizados.

Dos imágenes tienen embeddings cercanos. ¿Qué puedes concluir?

- A El modelo las representa como similares.
- B Son idénticas píxel a píxel.
- C Pertenecen a una clase legalmente autorizada.

Respuesta A. Exacto: la cercanía expresa similitud según esa representación.

Una similitud alta significa que el modelo representa las imágenes de forma próxima. No demuestra identidad píxel a píxel ni una categoría concreta.

PCA: los vectores en color

En estas imágenes cada color representa el vector que DINO saca de esa zona. Se pintan en cuatro pasos.

1 **Un vector por parche**

La imagen se parte en parches de 16 × 16 píxeles y el modelo devuelve un vector por parche, de 768 números en un ViT base. Una imagen de 224 × 224 da 196 vectores.

2 **PCA sobre todos los parches**

Se juntan esos vectores y se buscan las tres direcciones en las que más varían. Es álgebra pura, sin entrenamiento ni etiquetas.

3 **Tres números por parche**

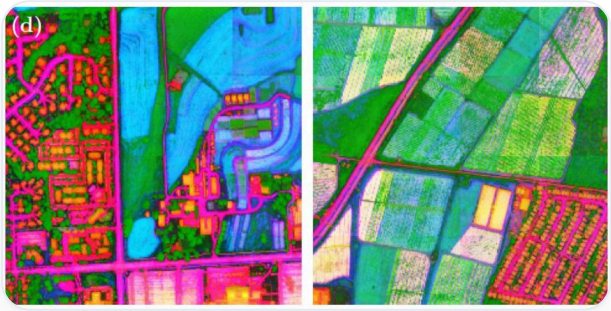
Cada parche se proyecta sobre esas tres direcciones. Los 768 números se quedan en tres, que es justo lo que cabe en un color.

4 **Los tres números se pintan**

Se reescalan a 0-255 y se usan como rojo, verde y azul. El parche vuelve a su sitio en la rejilla y aparece la imagen de colores.

Por qué el ala de un avión sale del mismo color que el ala de un pájaro. El color no lo decide el píxel, lo decide el vector. Dos parches con vectores parecidos reciben proyecciones parecidas y, por tanto, el mismo color, aunque uno sea metal gris y el otro plumas marrones.

Que el modelo los deje cerca significa que ha aprendido algo sobre ellos sin que nadie se lo dijera: los dos son superficies alargadas que salen de un cuerpo y lo sostienen. Igual caen juntas la rueda de un coche y la de una bicicleta, o el cielo y el mar.



Aquí las carreteras salen magenta en las dos imágenes, y la vegetación verde. Ningún color estaba definido de antemano.

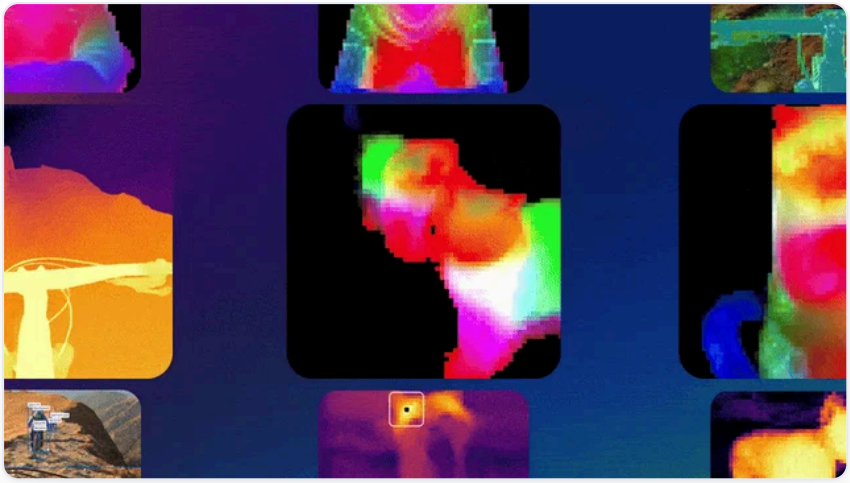
> Para qué sirve más allá de la ilustración: es la forma rápida de comprobar si un modelo entiende tus imágenes antes de montar nada encima. Si al pintar el PCA de tus piezas la zona buena y la zona defectuosa salen de colores distintos, merece la pena probar ese backbone; si sale todo del mismo color, mira otro modelo u otra resolución. Es una exploración, no una medida de rendimiento.

Dos avisos prácticos. El primero: los colores no son comparables entre imágenes salvo que el PCA se ajuste con todas a la vez, porque cada ajuste elige sus propias direcciones y el signo de cada componente es arbitrario. Por eso las figuras publicadas suelen calcular el PCA sobre el conjunto entero de imágenes que enseñan. El segundo: la primera componente suele separar objeto y fondo más que partes concretas, así que es habitual usarla como máscara, quedarse solo con los parches del objeto y repetir el PCA sobre ellos para que las tres componentes se gasten en distinguir partes. El procedimiento no depende de DINO: se puede aplicar a las características densas de cualquier modelo, y lo que cambia de uno a otro es cuánta estructura aparece.



Fíjate en que el color de cada región se mantiene mientras la escena se mueve: el modelo no ha visto una sola etiqueta.

DINOv3: características densas



Cada parche conserva su representación entre fotogramas, que es lo que permite usarlas para correspondencias y seguimiento.

Cuándo usar (y cuándo no) DINO

DINO no sirve para todo. Estas son las señales a favor y en contra.

Úsalo si…

- No tienes etiquetas o son muy costosas de obtener.
- Las clases o categorías cambian frecuentemente y reentrenar es inviable.
- Necesitas features reutilizables para múltiples tareas (segmentación, anomalías, búsqueda…).
- Quieres descubrir patrones sin asumir taxonomía previa.
- Buscas robustez a cambios de dominio (iluminación, fondo, escala).

Evítalo si…

- Tienes muchas etiquetas fiables y un objetivo de clasificación bien definido (supervisado será mejor).
- Requieres tiempo real en edge con hardware muy limitado (DINO es más pesado que MobileNet).
- Necesitas explicabilidad detallada y reglas interpretables (modelos más simples o clásicos).
- El problema es extremadamente específico y las features generales no aportan (raro, pero posible).

Regla empírica: si tienes etiquetas abundantes y un objetivo fijo, entrena supervisado. Si no, prueba primero los vectores de DINO con tus imágenes.

Detección de anomalías en inspección

En una línea de producción casi todas las piezas salen bien. Hay miles de fotos buenas, muy pocas malas, y cada defecto nuevo puede ser distinto del anterior.

Por qué no basta un detector

Un detector como los del tema 1 necesita muchos ejemplos de cada defecto. Aquí no los hay, y el defecto de mañana puede no parecerse a ninguno de los que tienes.

La idea

Se aprende cómo es una pieza buena y se marca lo que se aleja de eso. Para entrenar solo hacen falta fotos de piezas buenas.

Qué devuelve

Una puntuación por imagen, para aceptar o rechazar la pieza, y un mapa de calor que dice dónde está lo raro.

Hay dos tipos de anomalía. Las **estructurales** se ven en la superficie: arañazos, grietas, manchas, un hilo suelto. Las **lógicas** son piezas bien hechas pero mal montadas: falta un tornillo, sobra un cable, un componente va girado. Las segundas son más difíciles para un método que compara trozos pequeños.

Datasets de anomalías: MVTec AD

Los métodos se comparan con datasets que imitan una línea real: un entrenamiento solo con piezas buenas y un test con los defectos marcados píxel a píxel.

MVTec AD (2019)

El de referencia: 5.354 fotos de 15 categorías. Diez son objetos (tornillo, botella, cápsula, cable, transistor, avellana…) y cinco son texturas (alfombra, madera, cuero, baldosa, rejilla).

- **Entrenamiento:** 3.629 fotos, todas buenas.
- **Test:** 1.725 fotos, buenas y con más de 70 tipos de defecto, cada uno con su máscara.

mvtec.com: MVTec AD

Otros que vais a encontrar

- **VisA** (Amazon, 2022): 10.821 fotos de 12 objetos, entre ellos cuatro tipos de placa de circuito; 1.200 tienen defecto. github.com/amazon-science/spot-diff
- **MVTec AD 2** (2025): más de 8.000 fotos en 8 escenarios difíciles, con objetos transparentes, cambios de luz y piezas muy variables. mvtec.com: MVTec AD 2
- **MVTec LOCO AD**: anomalías estructurales y también lógicas, piezas que faltan, sobran o están mal colocadas. mvtec.com: MVTec LOCO AD

Para probar un método en clase o en casa, descarga una categoría de MVTec AD (la de alfombra o la de tornillos) y entrena solo con la carpeta de buenas. Ojo con la licencia: MVTec AD es CC BY-NC-SA 4.0 y no permite uso comercial (VisA es CC BY 4.0). Para un producto hacen falta fotos de tu propia línea.

PatchCore con DINO, paso a paso

PatchCore (Amazon, 2022) compara cada trozo de la imagen nueva con un banco de trozos de piezas buenas. Si un trozo no se parece a ninguno, ahí hay algo raro.

1

Fotos de piezas buenas

Para una primera prueba, entre 10 y 50, con la misma cámara, distancia y luz que en producción. Si hay variantes, lotes o turnos distintos, harán falta más.

2

Un vector por parche

DINOv2 o DINOv3 parte cada foto en parches de 14 o 16 píxeles y da un vector por parche. Se usan los parches, no el vector que resume la imagen entera.

3

El banco de parches buenos

Se guardan los vectores de todas las fotos buenas. Para que ocupe menos se queda un subconjunto que cubra todos los casos, entre el 1 y el 10 % (CoreSet).

4

La imagen nueva

Se parte igual y se saca un vector por parche.

5

Distancia al más parecido

Cada parche busca su vecino más cercano en el banco. Esa distancia es su puntuación: pequeña si se parece a algo bueno, grande si no se parece a nada.

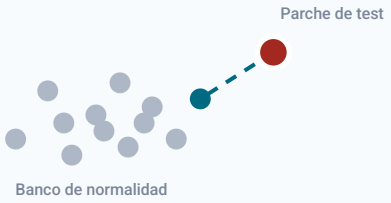
6

Mapa y decisión

Las puntuaciones de todos los parches forman el mapa de calor. La de la imagen es, simplificando, la del peor parche, y se compara con el umbral.

Distancia al banco de parches buenos

El banco solo guarda parches sin defectos: la puntuación es la distancia al más parecido que encuentre.



Esquema con dos dimensiones. DINO compara embeddings de varios cientos de dimensiones por parche.

Anomalía

Distancia al vecino

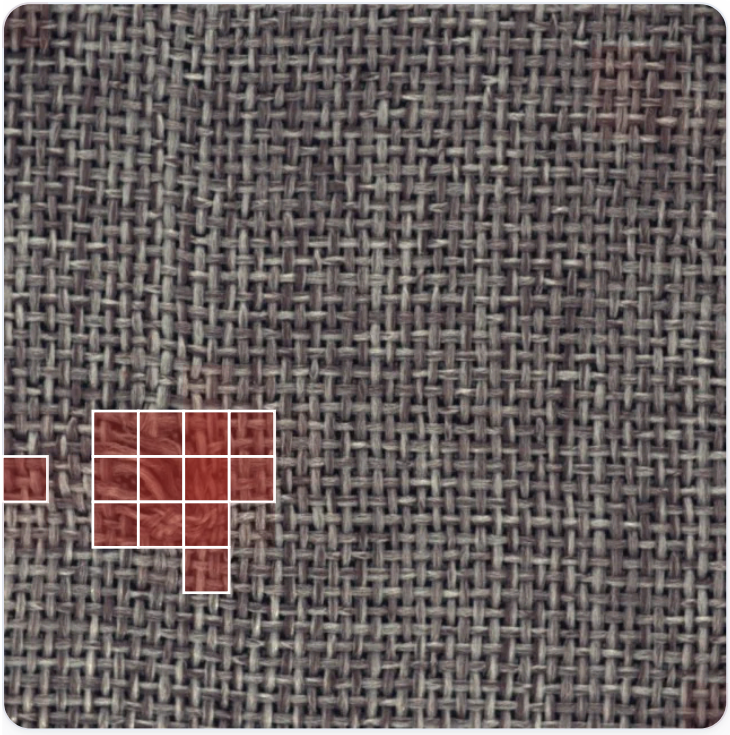
21,58

El parche queda a 21,58 de lo más parecido que hay en el banco, por encima del umbral 14,00.

Esta página recoge el estado con el parche al 55 % del recorrido. El banco no contiene ni un solo ejemplo de defecto: por eso el método sirve cuando los fallos son raros o desconocidos. Un parche se considera anómalo cuando ningún vector del banco se le parece lo suficiente, y el umbral se fija observando parches normales de validación, no adivinando. Con cada parche puntuado así, la imagen completa produce un heatmap y un score global.

Aceptar o rechazar la pieza

La misma alfombra, buena y con un defecto. Mueve el umbral y mira qué celdas se encienden en cada una.



Cada celda es un parche. El color de fondo es su puntuación; el borde marca las que superan el umbral. Puntuaciones precalculadas con un descriptor clásico, no con DINO.

Se rechaza

Celdas marcadas

13

Puntuación más alta

1,89

Qué pieza pasa por la cámara

Pieza con defecto

Pieza buena

El defecto queda marcado con 13 celdas, y la más alta llega a 1,89. La pieza sale de la línea.

> Así se monta una inspección real: el banco se llena con piezas buenas de la propia línea, el umbral se fija con un lote de validación mirando cuántas falsas alarmas aguanta el proceso, y el mapa sirve para enseñar al operario *dónde* está el problema, no solo que existe.

Esta página recoge el estado de la pieza defectuosa con umbral 1,30. Las puntuaciones están precalculadas con un descriptor clásico, no con DINO: cada celda de 35 × 35 píxeles se resume en siete estadísticas de intensidad y de gradiente, se normaliza con las celdas buenas y se puntúa por la distancia a la celda buena más parecida, que es exactamente la regla de PatchCore. Una celda de la pieza buena se compara con las demás celdas buenas y nunca consigo misma, así que su puntuación es honesta. El mapa se suaviza con una media de 3 × 3 para que un parche raro suelto no dispare la alarma. Con este descriptor la pieza buena llega como máximo a 1,28 y la defectuosa a 1,89: el margen existe pero es estrecho, y en cuanto el umbral baja de 1,28 la pieza buena ya enciende alguna celda y se para. Con características de DINOv2 el margen se abre, que es justamente el motivo de usarlas, y el cuaderno del tema lo comprueba con el mismo par de imágenes.

Cómo se mide un detector de anomalías

Aquí sí hay verdaderos negativos, las piezas buenas que pasan, así que se usan las métricas de clasificación del tema 1.

AUROC de imagen

Cuánto separa las piezas buenas de las malas, sin fijar ningún umbral. 1 es separación perfecta y 0,5 es tirar una moneda. PatchCore llega hasta el 99,6 % en MVTec AD, y otros métodos se le acercan; por eso se publicó MVTec AD 2, más difícil.

Métricas de píxel

El AUROC por píxel y el PRO miden si el mapa marca la zona del defecto. El PRO da el mismo peso a un defecto pequeño que a uno grande. Importan cuando el operario tiene que ver dónde está el fallo.

En la línea: el punto de trabajo

TPR a un TNR del 95 %: si aceptas parar 5 de cada 100 piezas buenas, qué parte de las malas detectas. Es la métrica operativa del tema 1, y la que entiende el jefe de planta.

El AUROC sirve para elegir método. El umbral se fija después con piezas buenas de validación, y en producción se cuentan dos cosas: cuántas buenas se paran (falsas alarmas) y cuántas malas se escapan. Para aceptar el sistema hace falta probarlo también con piezas malas reales, aunque no se usen para entrenar.

Qué hace falta para detectar una anomalía

Un vector por sí solo no dice si algo está bien o mal.

Quieres detectar una anomalía industrial con DINO. ¿Qué falta además del embedding?

- A Ejemplos normales y evaluación del umbral.
- B Una etiqueta textual para CLIP.
- C Solo más memoria GPU.

Respuesta A. Exacto: la distancia requiere referencias y una decisión validada.

Hace falta una referencia de normalidad y un criterio de distancia. Sin banco y sin umbral, un embedding es solo un punto en el espacio.

Sacar el vector de una imagen

El código base para buscar parecidos, agrupar imágenes o detectar anomalías.

```
import torch
from PIL import Image
from transformers import AutoImageProcessor, AutoModel

model_id = "facebook/dinov2-base"
processor = AutoImageProcessor.from_pretrained(model_id)
model = AutoModel.from_pretrained(model_id).eval()

def embedding(ruta):
    image = Image.open(ruta).convert("RGB")
    inputs = processor(images=image, return_tensors="pt")
    with torch.no_grad():
        salida = model(**inputs).last_hidden_state
    cls = salida[:, 0]                      # resumen global de la imagen
    return torch.nn.functional.normalize(cls, dim=-1)

a, b = embedding("pieza_1.jpg"), embedding("pieza_2.jpg")
print("similitud:", float(a @ b.T))
```

- El **token CLS** resume la imagen entera. Los demás, `salida[:, 1:]`, son un vector por parche y sirven para lo espacial.
- Con DINOv3, el que usa el cuaderno, los parches empiezan después de cuatro tokens de registro: `salida[:, 5:]`. La copia oficial de Meta pide acceso; el cuaderno usa la de `timm` en el Hub (`timm/vit_small_patch16_dinov3.lvd1689m`), con los mismos pesos y sin permisos.
- **Normaliza siempre** antes de comparar: la similitud coseno mide dirección, no longitud.
- El modelo va congelado. No hay entrenamiento aquí, solo extracción.

Montar un detector de anomalías

Los pasos de PatchCore llevados a tu línea, sin ningún ejemplo de defecto.

1. **Reúne entre 10 y 50 imágenes normales**

   De la misma cámara, la misma distancia y la misma luz que vas a tener en producción. La variabilidad que no esté aquí se marcará como anomalía.

2. **Extrae embeddings por parche y normaliza**

   Usa los tokens de parche, no solo el CLS, y normaliza con L2. Sácalos de una capa intermedia: en la última, cada vector mezcla información de toda la imagen y un defecto altera también los parches sanos, de modo que el umbral deja de separar.

3. **Construye el banco con un CoreSet del 1 al 10 %**

   No guardes todos los vectores: quédate con un subconjunto que cubra el espacio. Reduce memoria y acelera la búsqueda sin perder cobertura.

4. **Fija el umbral con imágenes normales de validación**

   Puntúa unas cuantas imágenes buenas que no estén en el banco y coloca el umbral por encima de su distancia máxima, con margen. No lo adivines.

5. **Puntúa y revisa el heatmap, no solo el número**

   El máximo por parche da el score de la imagen; el mapa completo te dice si el modelo mira donde tú mirarías.

> **Señal de alarma:** si el heatmap se enciende en el fondo o en los bordes en vez de en la pieza, el problema es el encuadre o la iluminación, no el modelo.

Antes de usar un banco de normalidad

Seis comprobaciones antes de ponerlo en la línea.

- [ ] **Las imágenes normales cubren la variación real**
  Turnos, lotes, luz de mañana y de tarde. Lo que falte se marcará como anomalía.
- [ ] **Encuadre estable**
  Si la pieza cambia de posición entre fotos, la distancia sube sin que haya defecto.
- [ ] **Umbral fijado con validación**
  Con imágenes buenas apartadas, no con las del propio banco.
- [ ] **Heatmap revisado a ojo**
  Que se encienda donde está el defecto y no en el fondo.
- [ ] **Falsos positivos contados**
  Cuántas piezas buenas estás parando. Ese es el coste que notará producción.
- [ ] **Plan de actualización**
  Qué haces cuando cambie el proveedor, el lote o la iluminación.

Cuaderno 4: DINO

Los ejercicios de este tema están en el cuaderno.

1. Qué devuelve el modelo
2. Parecido entre imágenes con el token CLS
3. Correspondencias entre parches
4. PCA en color
5. Anomalías con la alfombra
6. Ejercicios

Abrir en Colab ↗

En Colab: Archivo, Guardar una copia en Drive, y Entorno de ejecución, GPU T4.